SE203 – Software Engineering

Behavioral Models

(Use-case, Activity, State, Sequence, Communication)

What is UML and Why we use UML?

- UML → “Unified Modeling Language”
- Language: express idea, not a methodology

- Modeling: Describing a software system at a high level of abstraction

- Unified: UML has become a world standard

  Object Management Group (OMG): www.omg.org

What is UML and Why we use UML?

- More description about UML:

- It is a industry-standard graphical language for specifying, visualizing, constructing, and documenting the artifacts of software systems

- The UML uses mostly graphical notations to express the OO analysis and design of software projects.

- Simplifies the complex process of software design

What is UML and Why we use UML?

- Why we use UML?

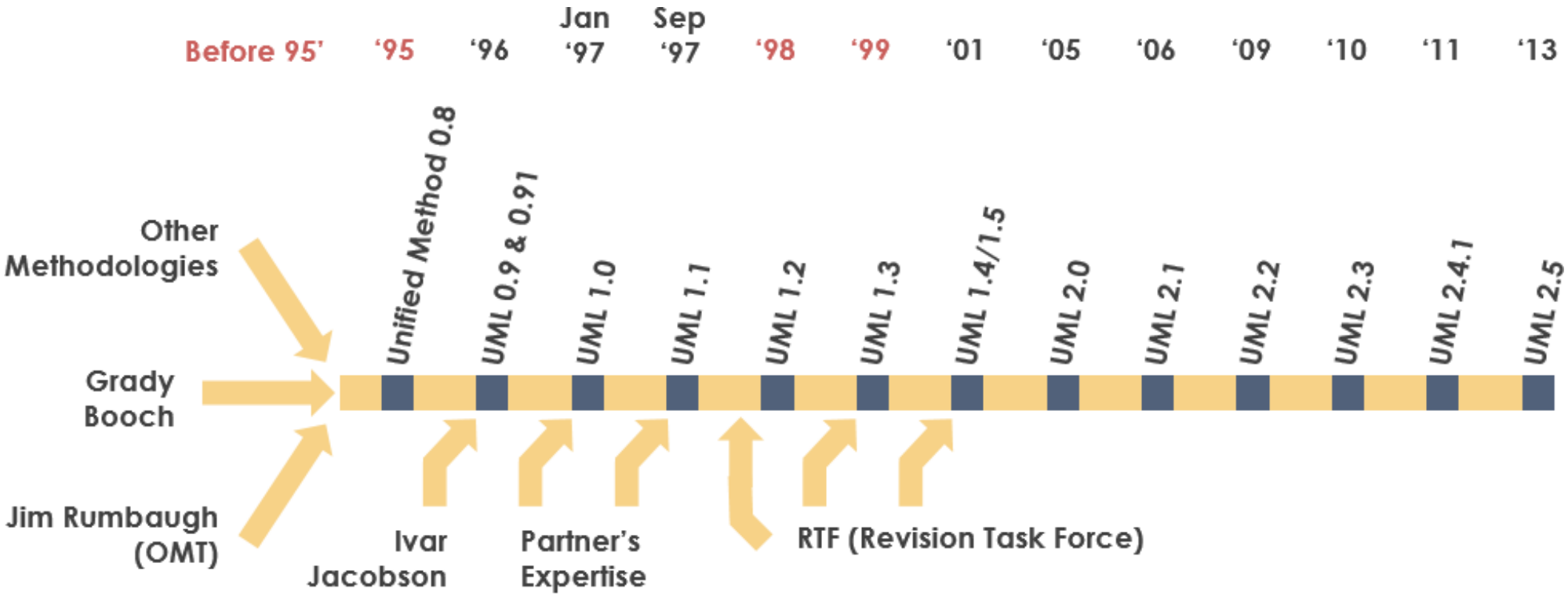
- Use graphical notation: more clearly than natural language (imprecise) and code (too detailed).

- Help acquire an overall view of a system.

- UML is *not* dependent on any one language or technology.

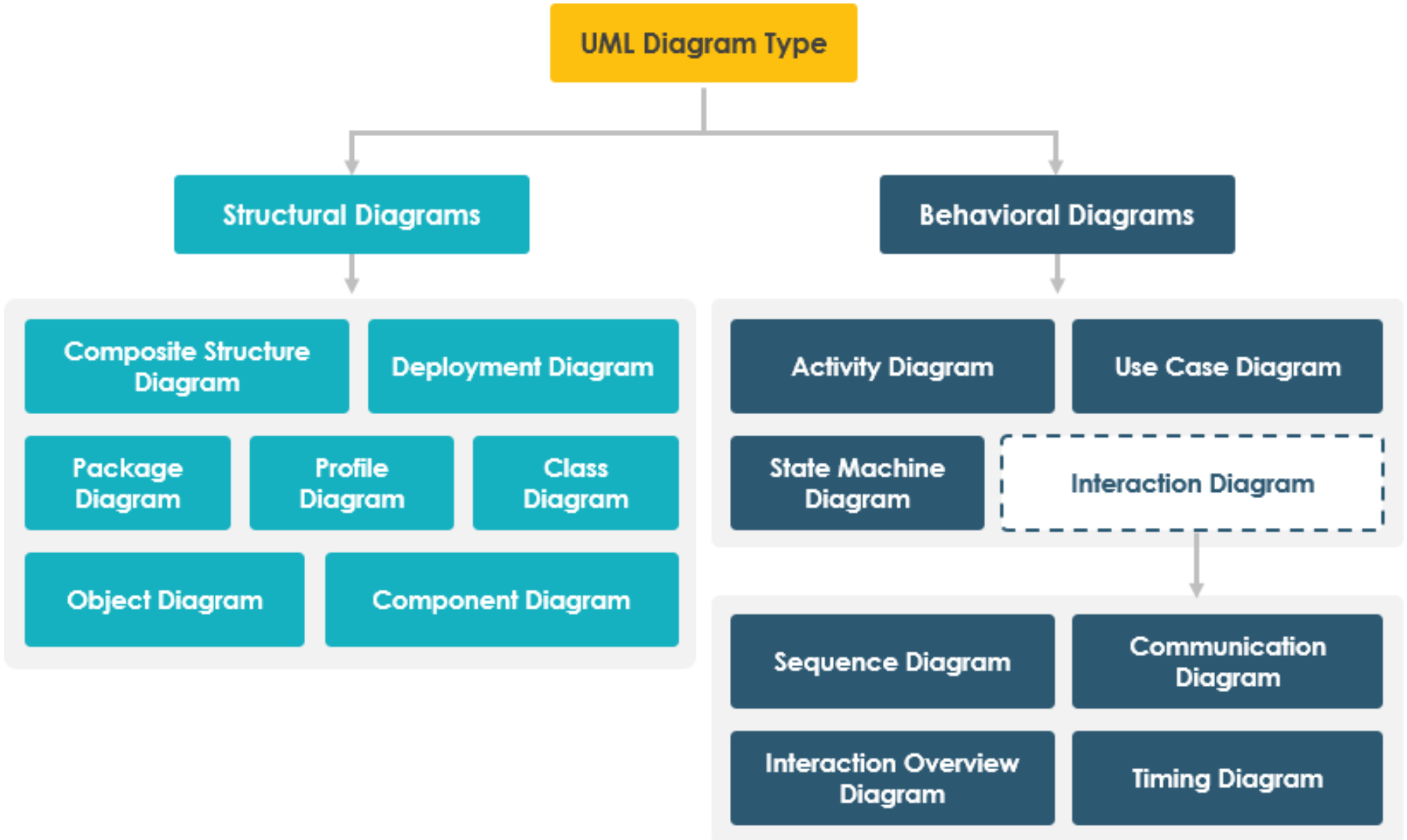
- UML moves us from fragmentation to standardization.

History of UML



Before 95' - Fragmentation ▶ 95' - Unification ▶ 98' - Standardization ▶ 99' - Industrialization

https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/

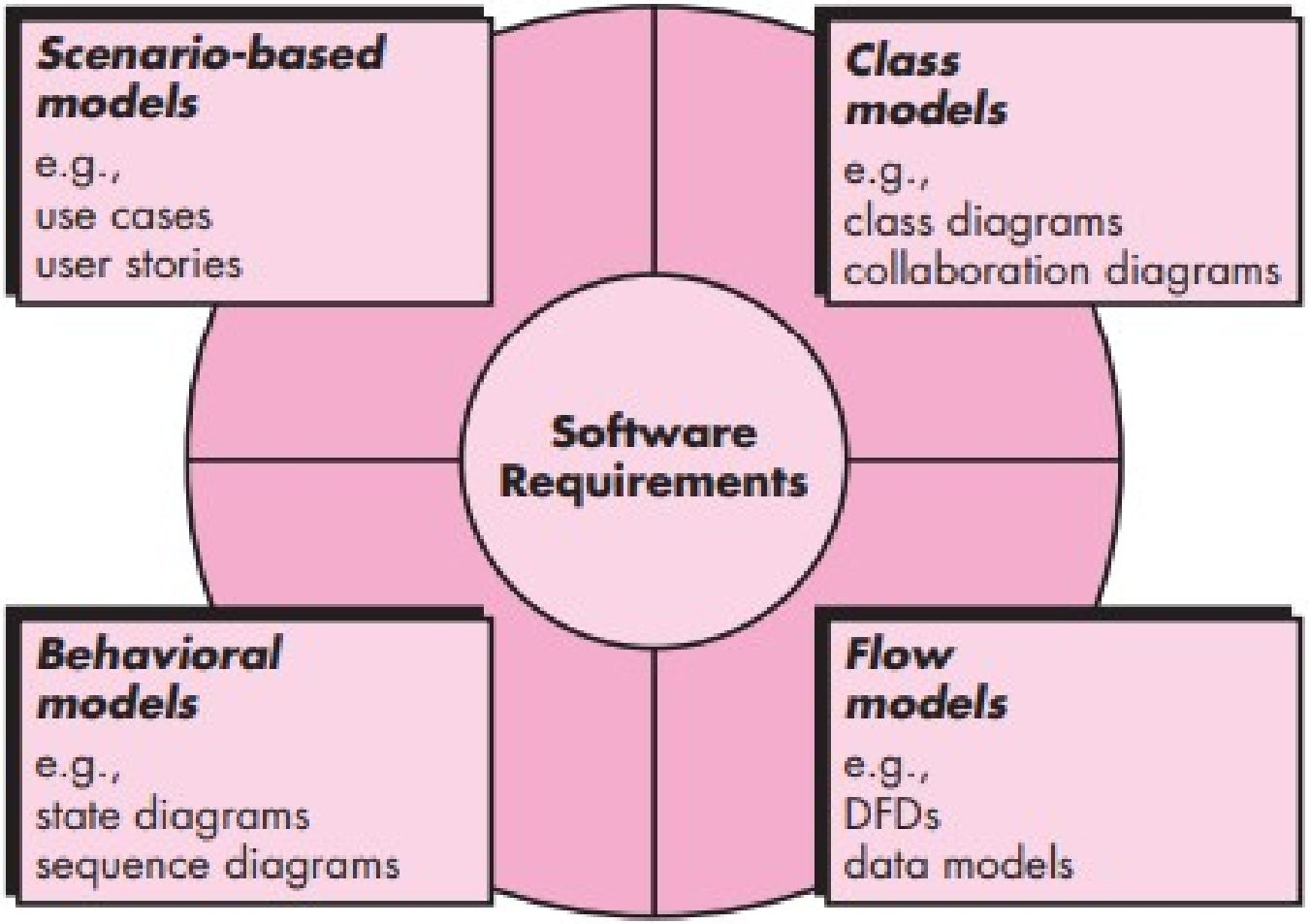


https://www.visual-paradigm.com/guide/uml-unified-modeling-language/behavior-vs-structural-diagram/

What UML Modeling tools

- List of UML tools http://en.wikipedia.org/wiki/List_of_UML_tools
- ArgoUML: http://argouml.tigris.org/
- Rational Rose (www.rational.com) by IBM
- UML Studio (http://www.pragsoft.com/) by Pragsoft Corporation:
- TogetherSoft Control Center; TogetherSoft Solo (http://www.borland.com/together/index.html) by Borland

Requirement Modelling



Use-Case Diagrams

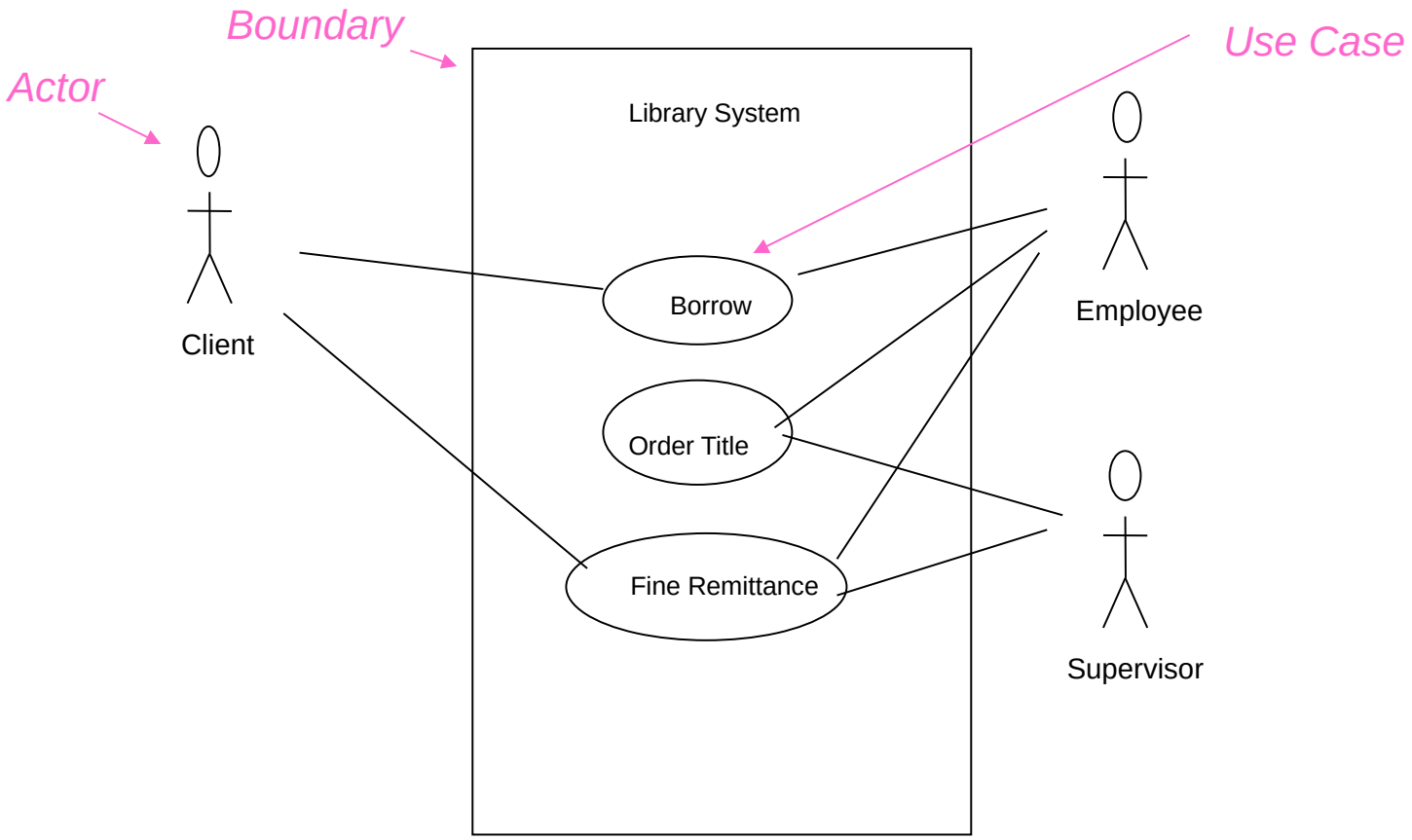
- A use-case diagram is a set of use cases
- A use case is a model of the interaction between
  - External users of a software product (actors) and
  - The software product itself
  - More precisely, an actor is a user playing a specific role

- describing a set of user **scenarios**
- capturing user requirements
- **contract** between end user and software developers

Use-Case Diagrams

- Components
  - A large box: *system boundary*
  - Stick figures outside the box: *actors*, both human and systems
  - Each oval inside the box: a use case that represents some major required functionality and its variant
  - A line between an actor and use case: the actor participates in the use case
- Use cases do not model all the tasks, instead they are used to specify user views of essential system behavior

Use-Case Diagrams

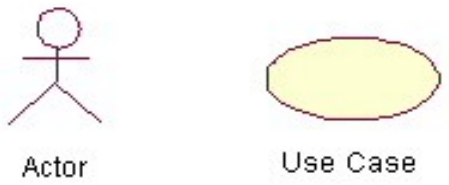


Use-Case Diagrams

- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.

- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.

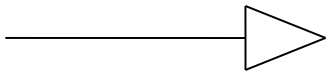
- **System boundary**: rectangle diagram representing the boundary between the actors and the system.



Use-Case Diagrams

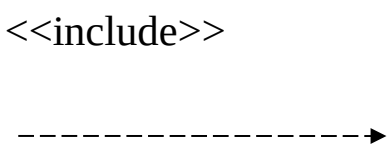
- Association:
  communication between an actor and a use case; Represented by a solid line.

- Generalization: relationship between one general use case and a special use case (used for defining special alternatives) Represented by a line with a triangular arrow head toward the parent use case.



Use-Case Diagrams

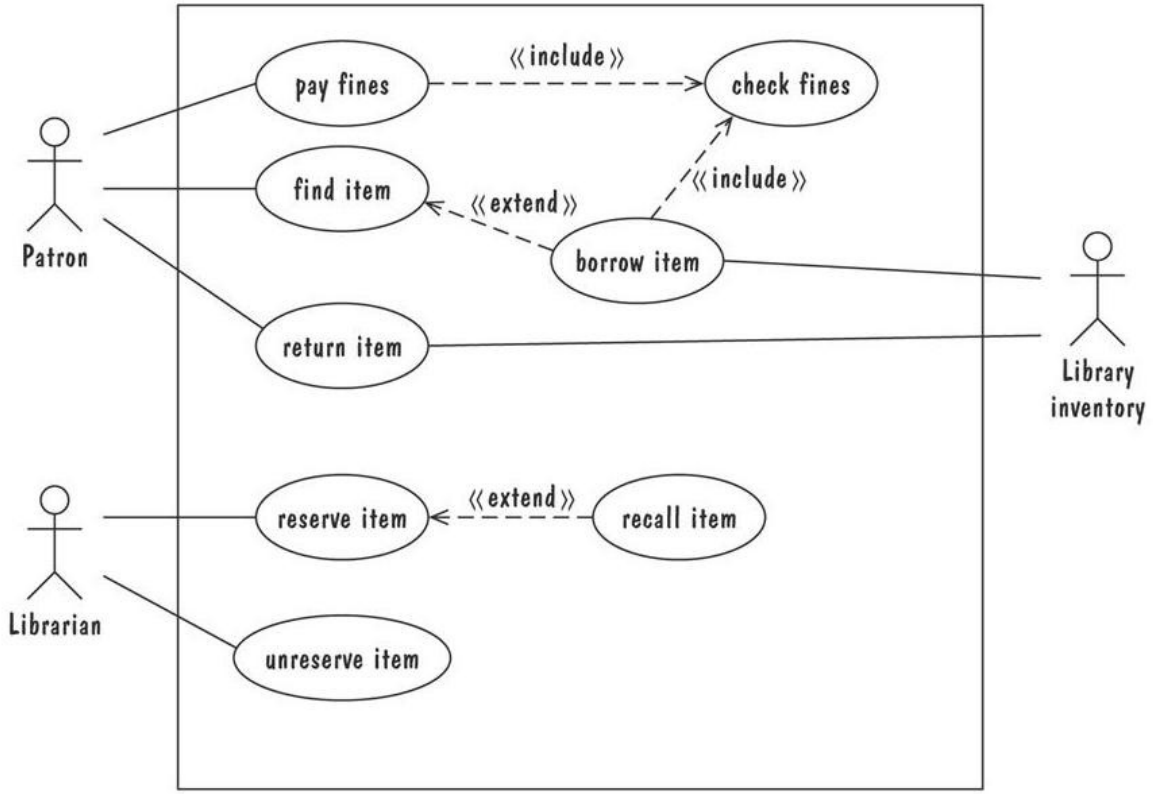
Include: a dotted line labeled <<include>> beginning at base use case and ending with an arrows pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use "include" in stead of copying the description of that behavior.



Extend: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares "extension points".

Use-Case Diagrams

- Library use cases including borrowing a book, returning a borrowed book, and paying a library fine



Use-Case Diagrams

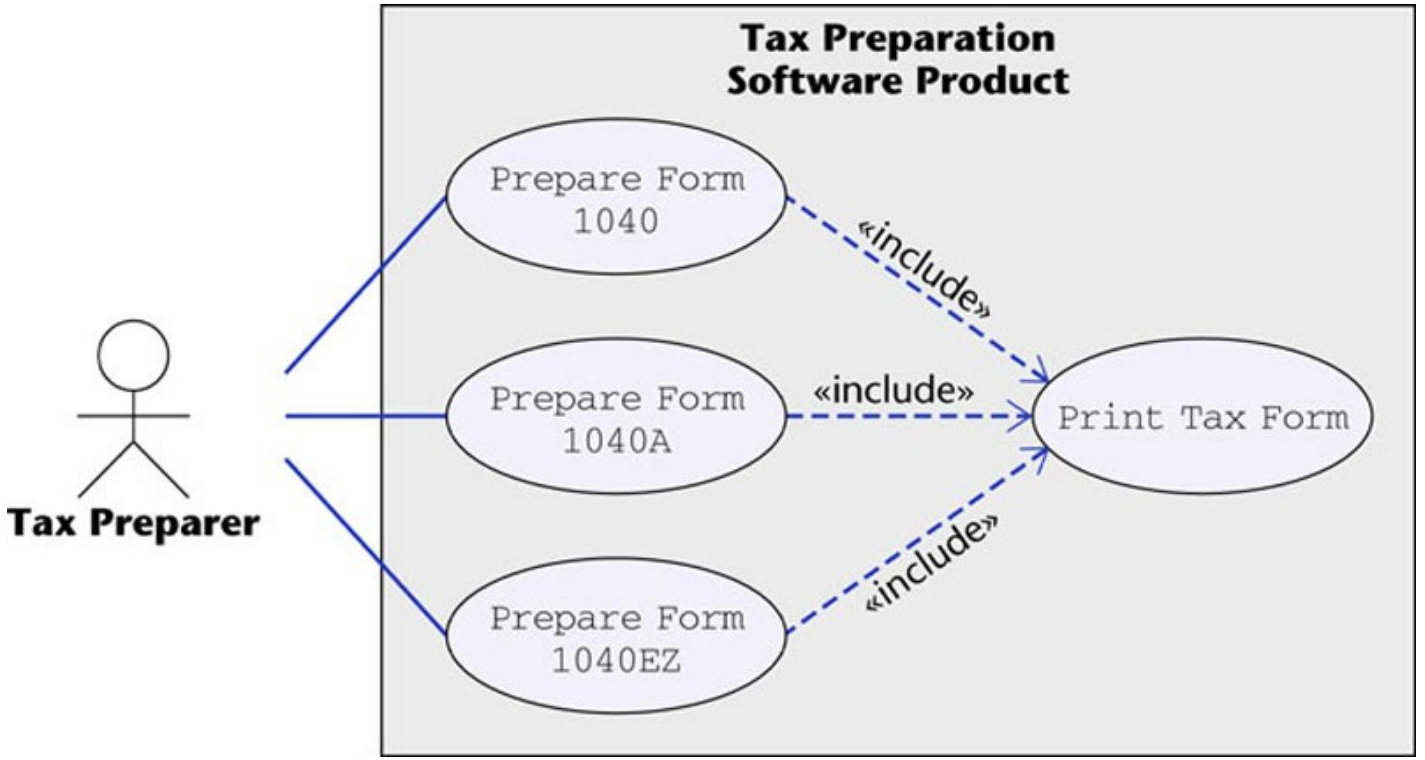
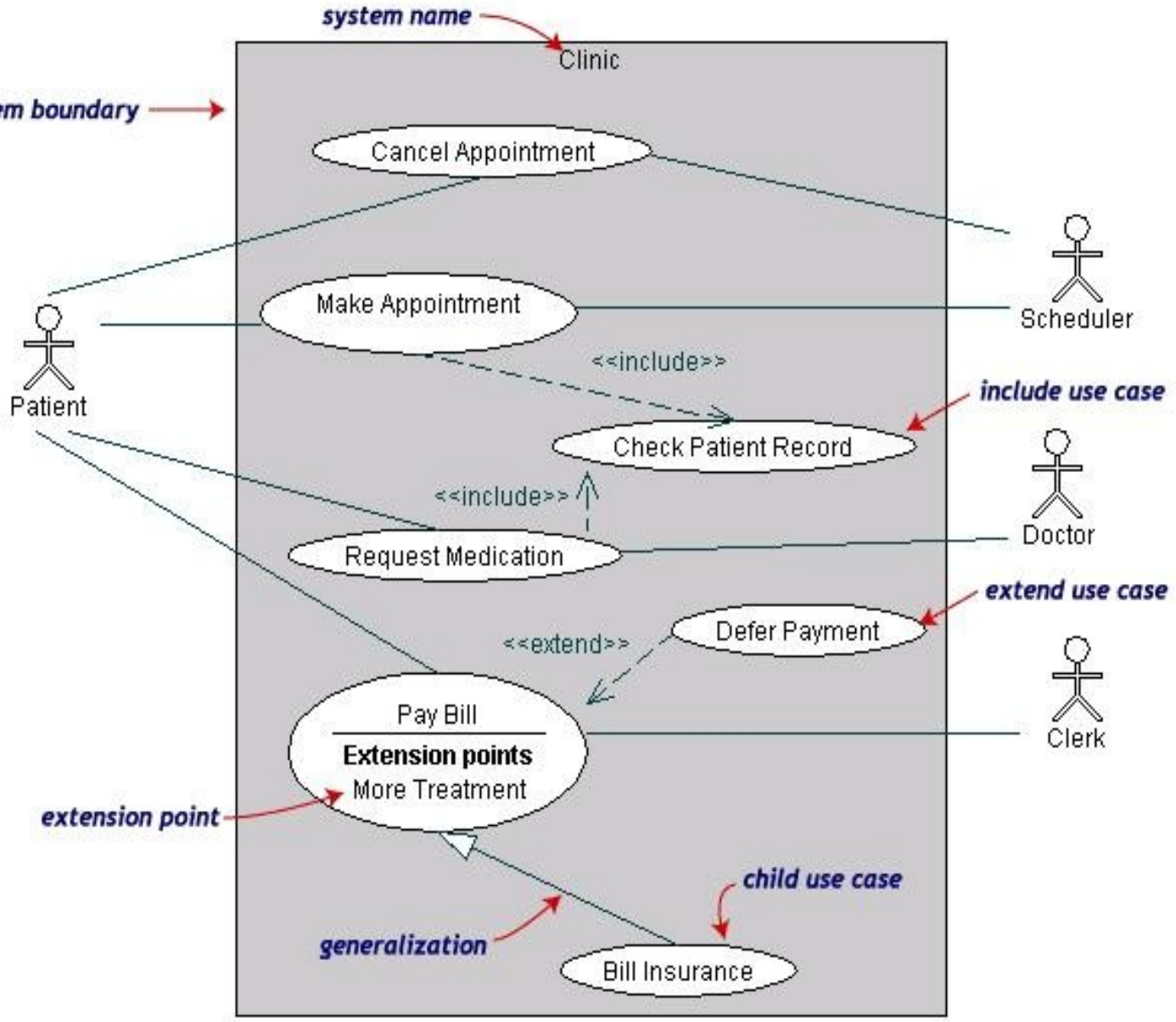


Figure 16.12

Use-Case Diagrams

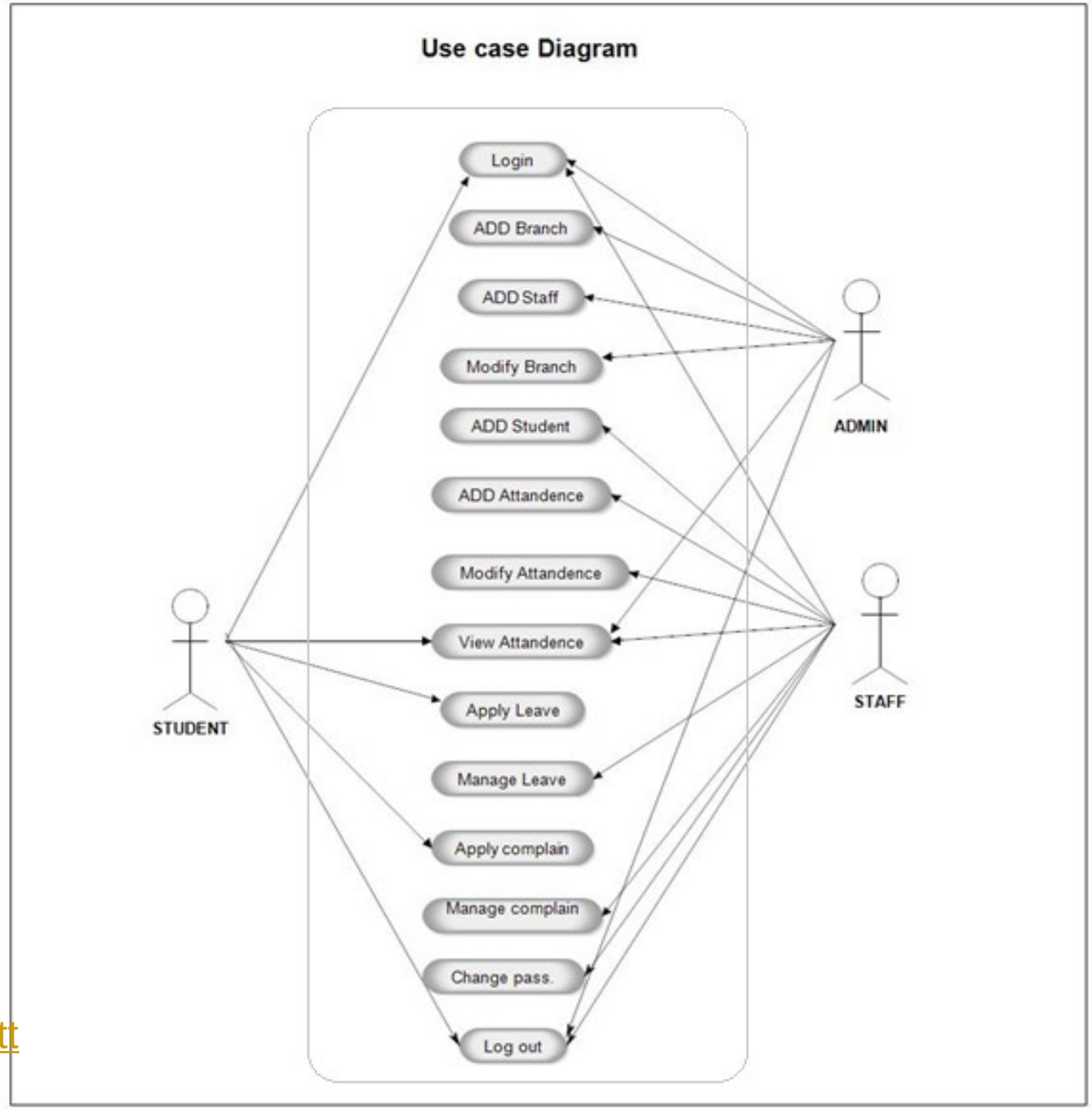
- Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask (include)
- The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)
- **Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)



(TogetherSoft, Inc)

Use-Case Diagrams

- Mark Attendance use cases



https://meeraacademy.com/use-case-diagram-student-attendance-management-system/

Activity Diagram

- Activity diagrams describe the workflow behavior of a system.
  - Activity diagrams are used in process modeling and analysis of during requirements engineering.
  - A typical business process which synchronizes several external incoming events can be represented by activity diagrams.
- They are most useful for understanding work flow analysis of synchronous behaviors across a process.

Activity Diagram

- Activity diagrams are used for
  - documenting existing process
  - analyzing new Process Concepts
  - finding reengineering opportunities.

- The diagrams describe the state of activities by showing the sequence of activities performed.
  - they can show activities that are conditional or parallel.

When to Use Activity Diagrams

- The main reason to use activity diagrams is to model the workflow behind the system being designed.

- Activity Diagrams are also useful for:
  - analyzing a use case by describing what actions need to take place and when they should occur
  - describing a complicated sequential algorithm
  - modeling applications with parallel processes

- Activity Diagrams should not take the place of interaction diagrams and state diagrams.

- Activity diagrams do not give detail about how objects behave or how objects collaborate.

Activity Diagram Concepts

- An activity is trigged by one or more events and activity may result in one or more events that may trigger other activity or processes.
- Events start from start symbol and end with finish marker having activities in between connected by events.
- The activity diagram represents the decisions, iterations and parallel/random behavior of the processing.
  - They capture actions performed.
  - They stress on work performed in operations (methods).

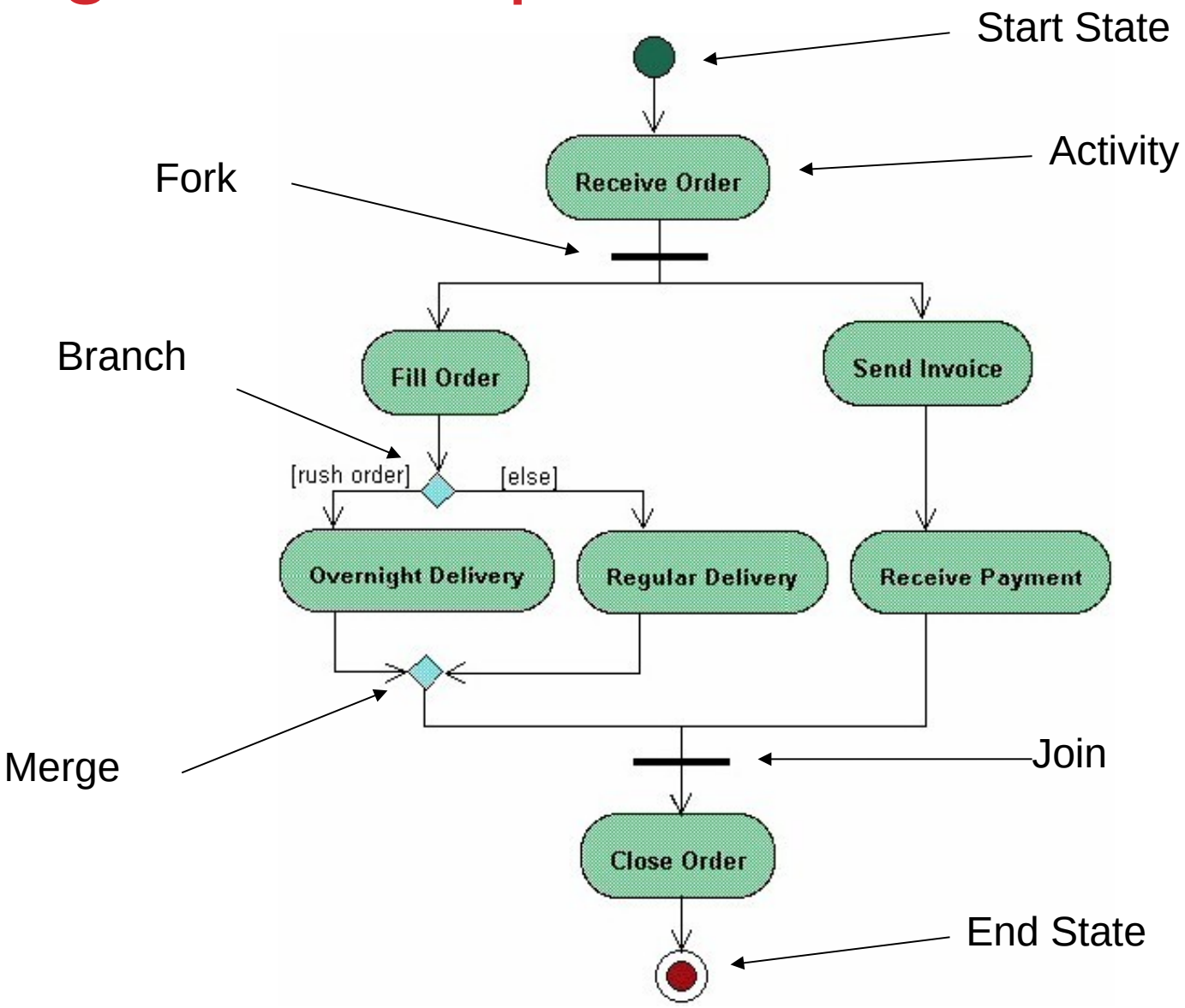
Components

- An ***activity*** is an ongoing, though interruptible, execution of a step in a workflow (such as an operation or transaction)
  - Represented with a rounded rectangle.
  - Text in the activity box should represent an activity (verb phrase in present tense).
- An ***event*** is triggered by an activity. It specifies a significant occurrence that has a location in time and space.
  - An instance of an event (trigger) results in the flow from one activity to another.
  - These are represented by directed straight lines emerging from triggering activity and ending at activity to be triggered. Label text for events should represent event but not the data involved.
- A ***decision*** may be shown by labeling multiple output transitions of an activity with different guard conditions.
  - For convenience a stereotype is provided for a decision: the traditional diamond shape, with one or more incoming arrows and with two or more outgoing arrows, each labeled by a distinct guard condition with no event trigger.

How to Draw an Activity Diagram

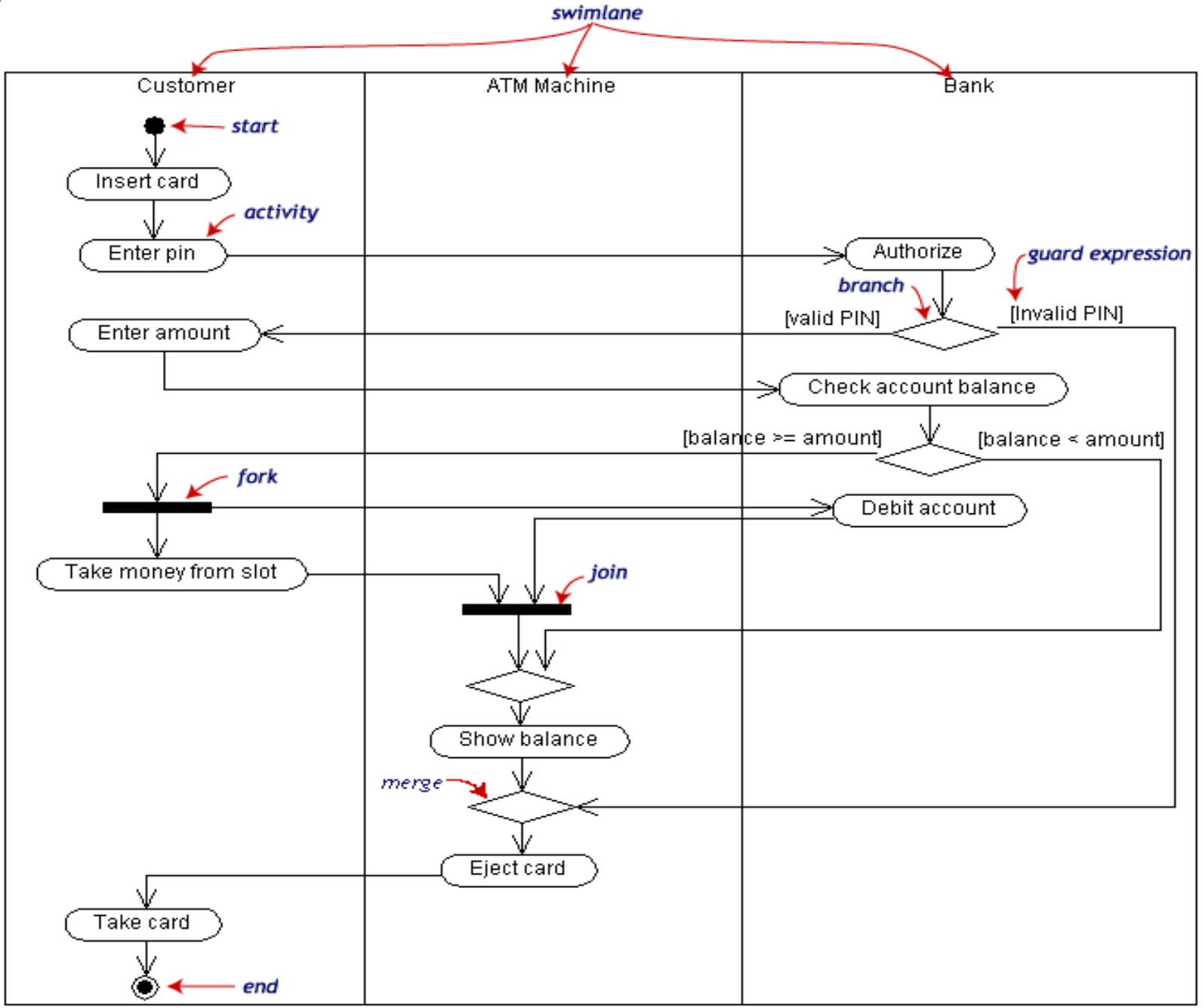
- Diagrams are read from top to bottom and have branches and forks to describe conditions and parallel activities.
  - A fork is used when multiple activities are occurring at the same time.
  - A branch describes what activities will take place based on a set of conditions.
  - All branches at some point are followed by a merge to indicate the end of the conditional behavior started by that branch.
  - After the merge all of the parallel activities must be combined by a join before transitioning into the final activity state.

Activity Diagram Example



Activity Diagram to Swinlane Diagram

- Withdraw money from a bank account through an ATM



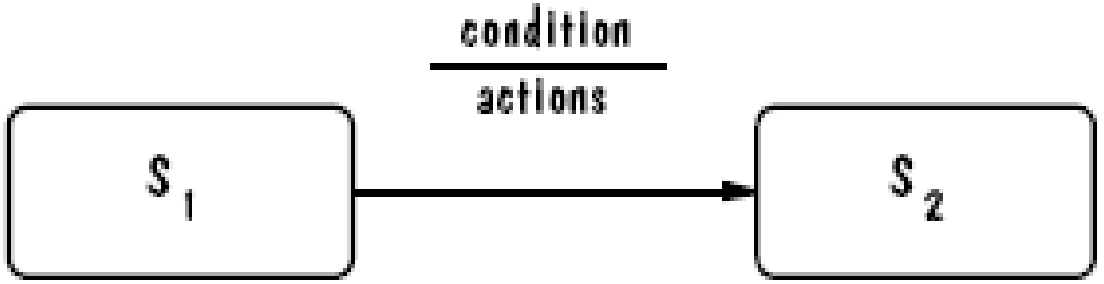
Disadvantages

- A disadvantage of activity diagrams is that they do not explicitly present which objects execute which activities, and the way that the messaging works between them.
  - Labeling of each activity with the responsible object can be done.
  - It is useful to draw an activity diagram early in the modeling of a process, to help understand the overall process.

- Then interaction diagrams can be used to help you allocate activities to classes.

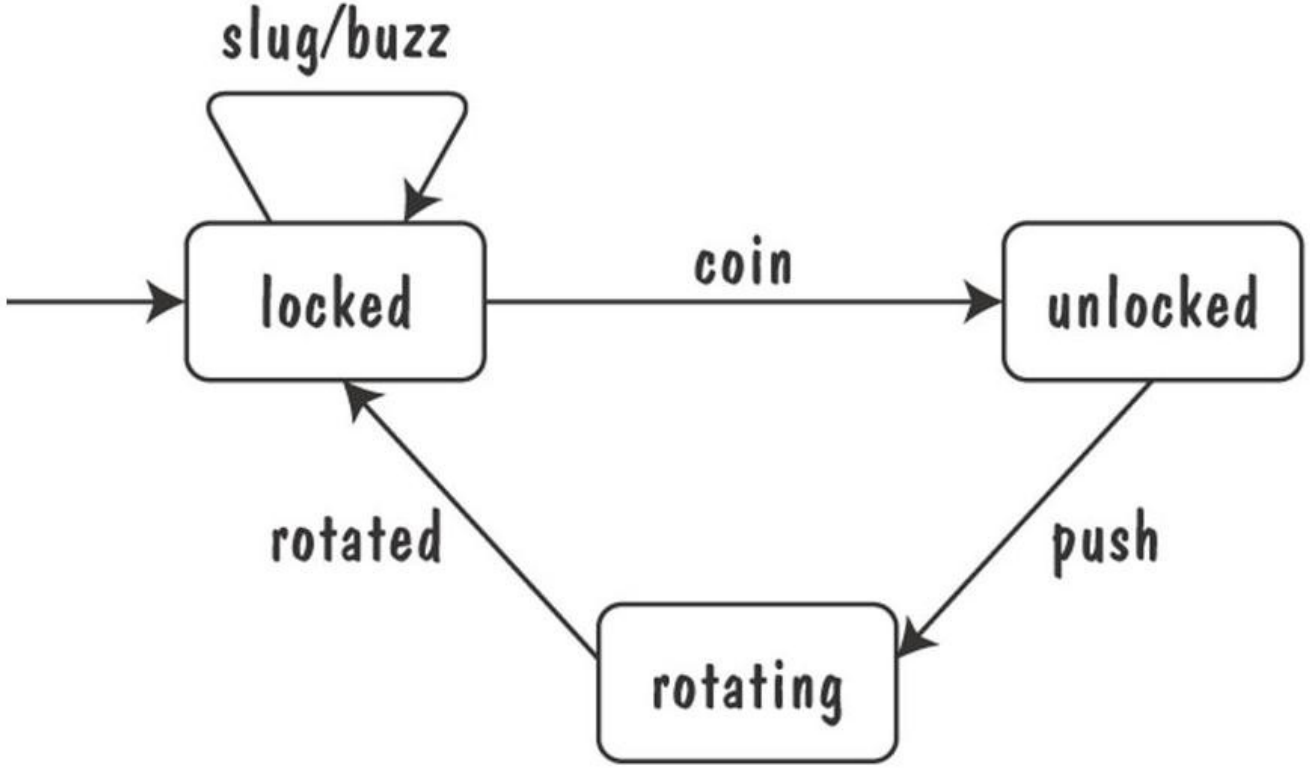
State Diagrams

- A graphical description of all dialog between the system and its environment
  - Node (*state*) represents a stable set of conditions that exists between event occurences
  - Edge (*transition*) represents a change in behavior or condition due to the occurrence of an event
- Useful both for specifying dynamic behavior and for describing how behavior should change in response to the history of events that have already occurred



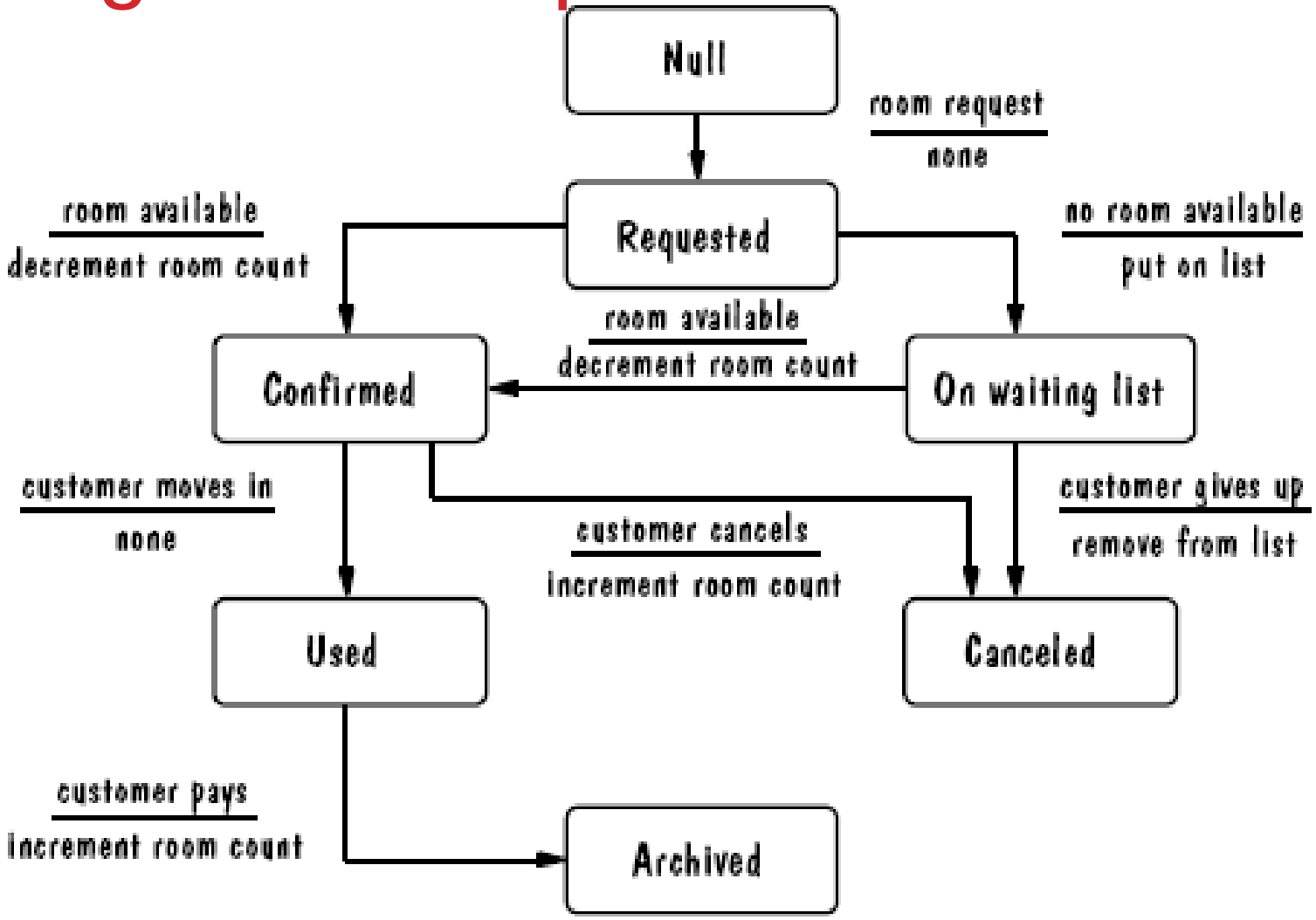
State Diagrams (Contd.)

- Finite state machine model of the tunstile problem



UML Statechart Diagram Example

- The UML state diagram for `Hotel Reservation`

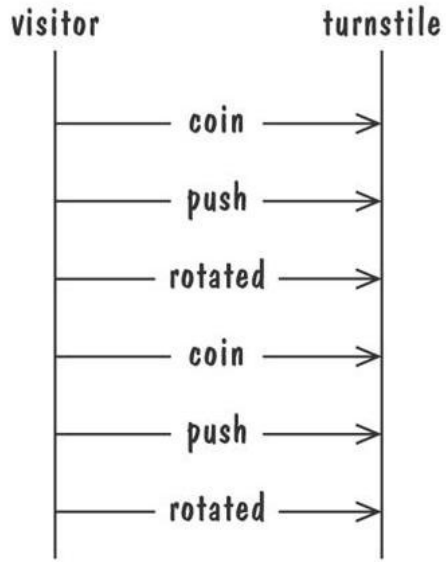


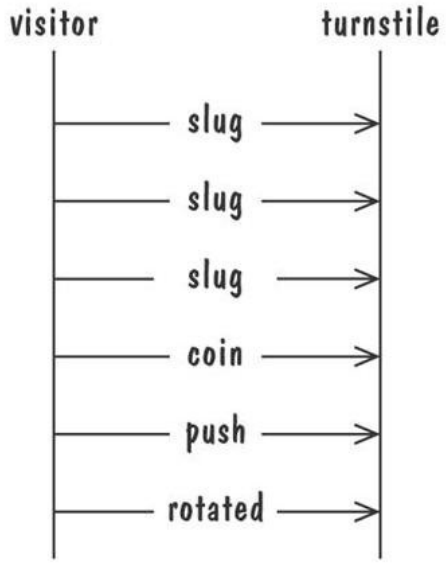
EVENT TRACES

- A graphical description of a sequence of events that are exchanged between real-world entities
  - *Vertical line*: the timeline of distinct entity, whose name appear at the top of the line
  - *Horizontal line*: an event or interaction between the two entities bounding the line
  - Time progresses from top to bottom
- Each graph depicts a single trace, representing one of several possible behaviors
- Traces have a semantic that is relatively precise, simple and easy to understand

EVENT TRACES

- Graphical representation of two traces for the turnstile pr
  - trace on the left represents typical behavior
  - trace on the right shows exceptional behavior

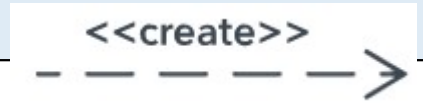




SEQUENCE DIAGRAM

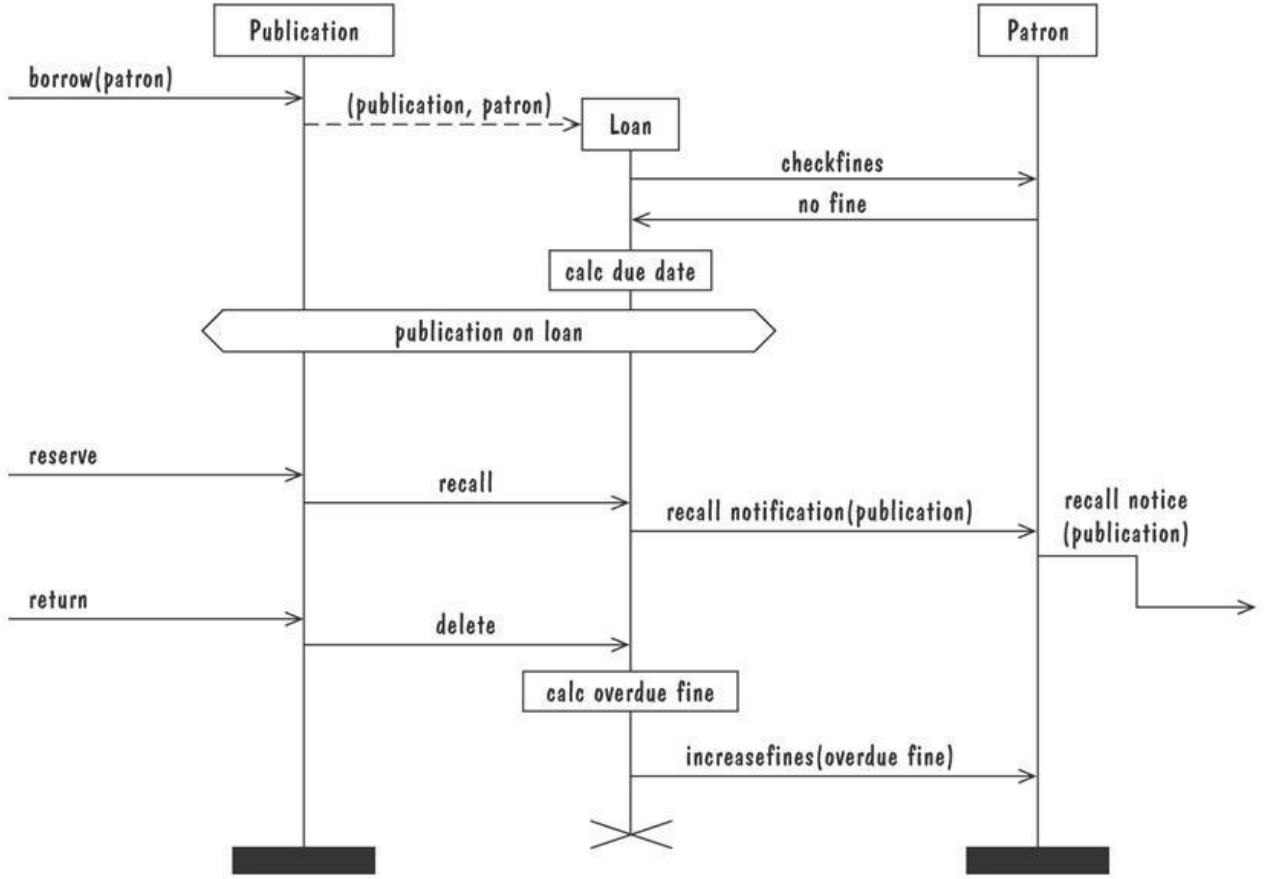
- An enhanced event-trace notation, with facilities for creating and destroying entities, specifiying actions and timers, and composing traces
  - *Vertical line* represents a participating entity
  - *A message* is depicted as an arrow from the sending entity to the receiving entity
  - *Actions* are specified as labeled rectangles positioned on an entity's execution line
  - *Conditions* are important states in an entity's evolution, represented as labeled hexagon

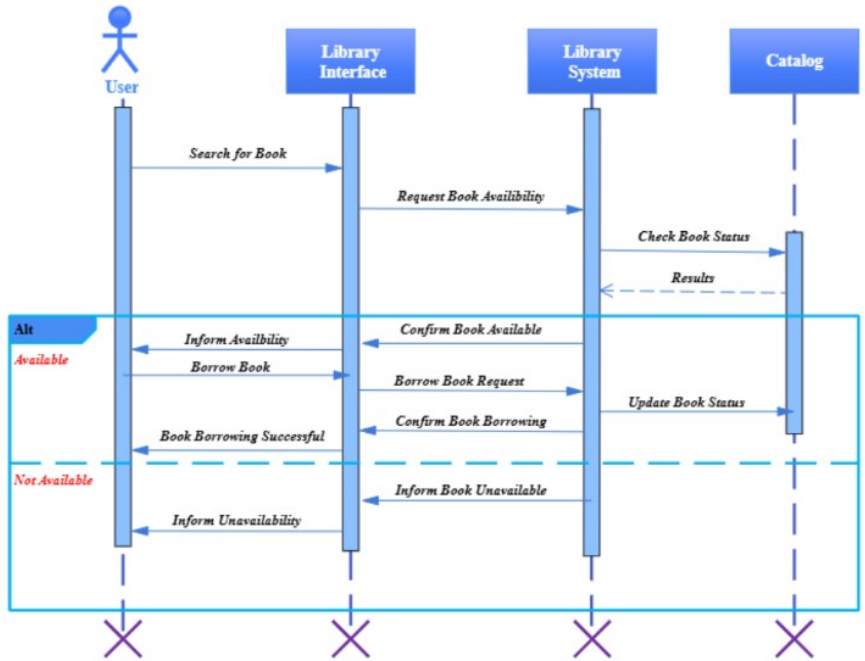
SEQUENCE DIAGRAM: COMMON MESSAGE SYMBOLS

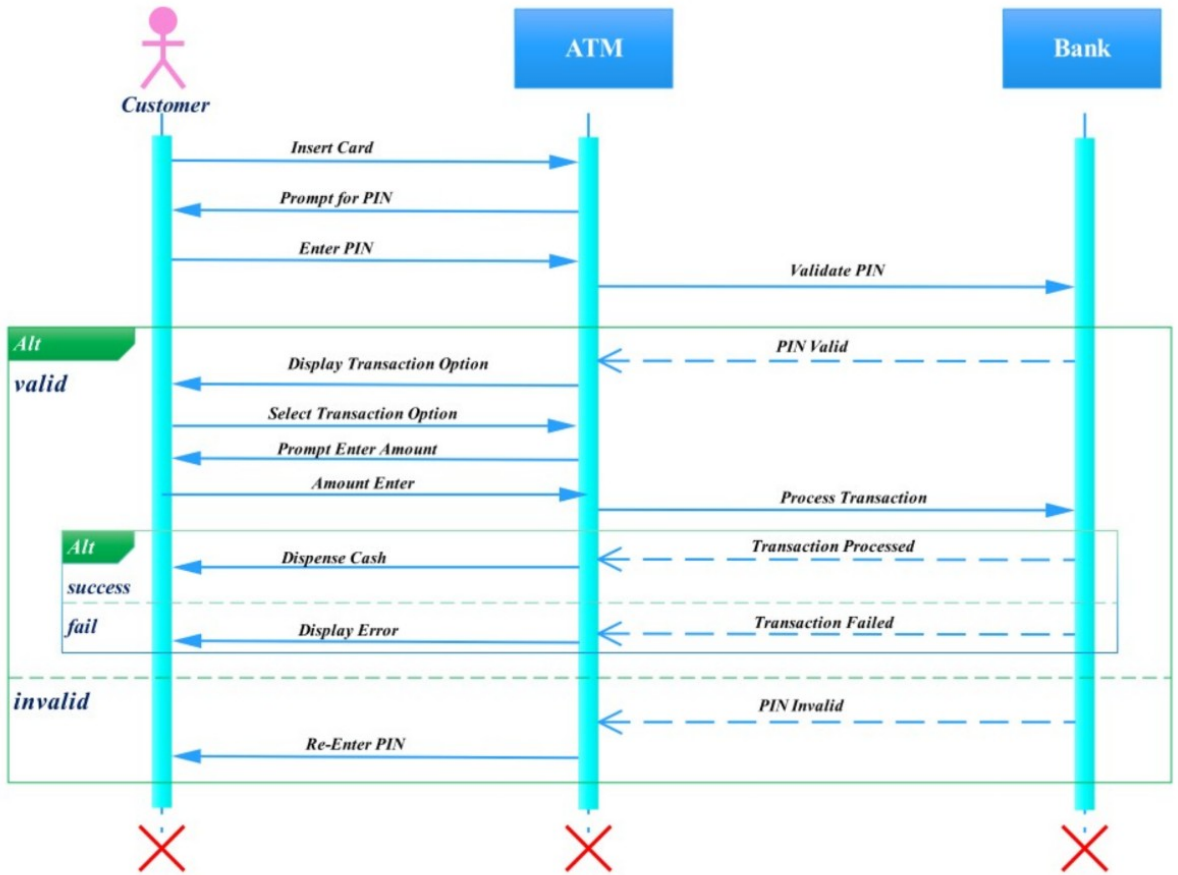
Symbol	Name	Description
	Synchronous message symbol	Represented by a solid line with a solid arrowhead. This symbol is used when a sender must wait for a response to a message before it continues. The diagram should show both the call and the reply.
	Asynchronous message symbol	Represented by a solid line with a lined arrowhead. Asynchronous messages don't require a response before the sender continues. Only the call should be included in the diagram.
	Asynchronous return message symbol	Represented by a dashed line with a lined arrowhead.
	Asynchronous create message symbol	Represented by a dashed line with a lined arrowhead. This message creates a new object.
	Reply message symbol	Represented by a dashed line with a lined arrowhead, these messages are replies to calls.
	Delete message symbol	Represented by a solid line with a solid arrowhead, followed by an X. This message destroys an object.

SEQUENCE DIAGRAM

- Message sequence chart for library loan transaction




User
Library Interface
Library System
Catalog
Search for Book
Request Book Availibility
Check Book Status
Results
Alt
Available
Inform Availbility
Confirm Book Available
Borrow Book
Borrow Book Request
Update Book Status
Confirm Book Borrowing
Book Borrowing Successful
Not Available
Inform Book Unavailable
Inform Unavailability


Customer
ATM
Bank
Insert Card
Prompt for PIN
Enter PIN
Validate PIN
Alt
valid
PIN Valid
Display Transaction Option
Select Transaction Option
Prompt Enter Amount
Amount Enter
Process Transaction
Alt
success
Dispense Cash
Transaction Processed
fail
Display Error
Transaction Failed
invalid
PIN Invalid
Re-Enter PIN

COMMUNICATION DIAGRAM

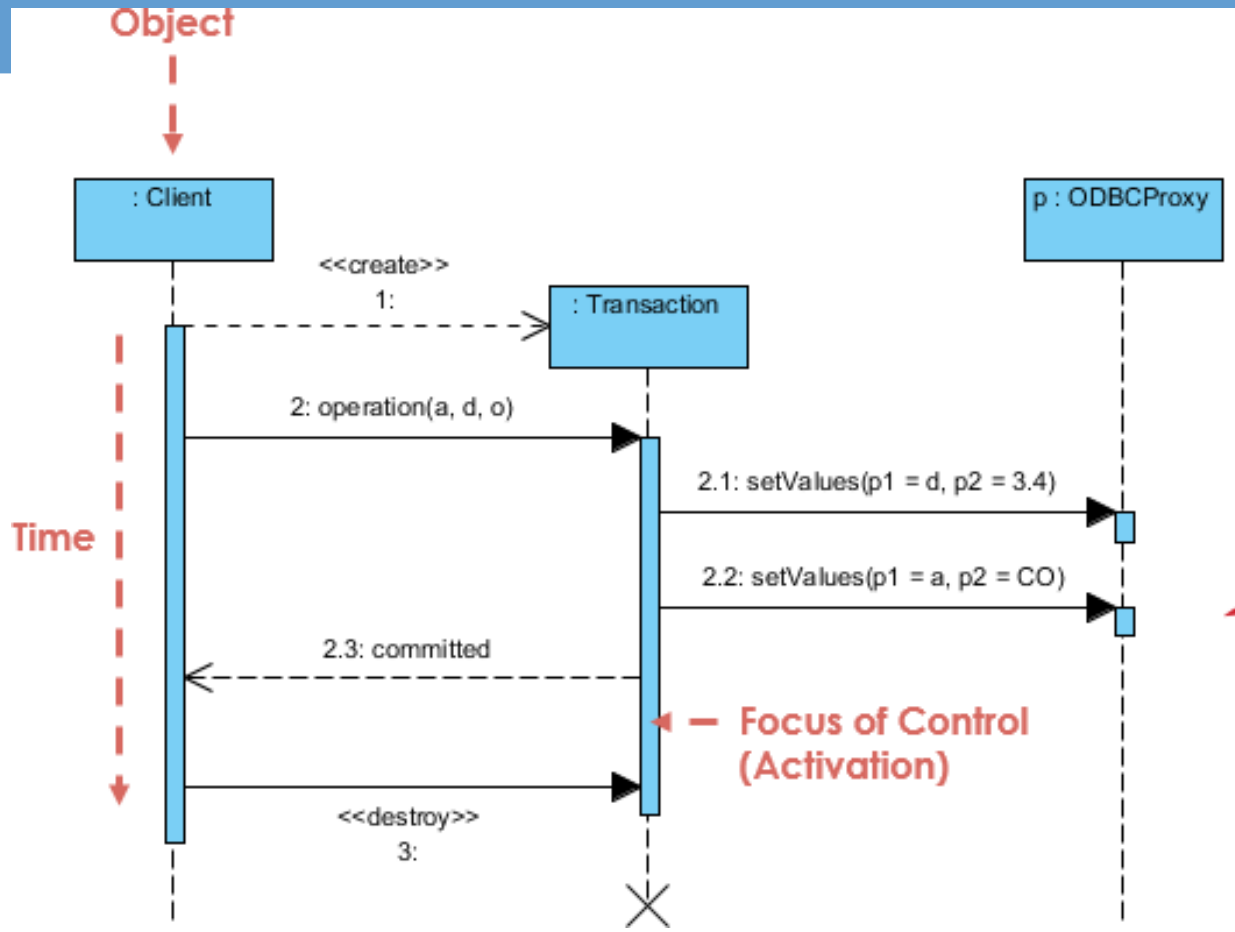
- A communication diagram offers the same information as a sequence diagram, but while a sequence diagram emphasizes the time and order of events, a communication diagram emphasizes the messages exchanged between objects in an application.
- Sequence diagrams can fall short of offering the "big picture.”
- This is where communication diagrams come in and offer that broader perspective within a process.

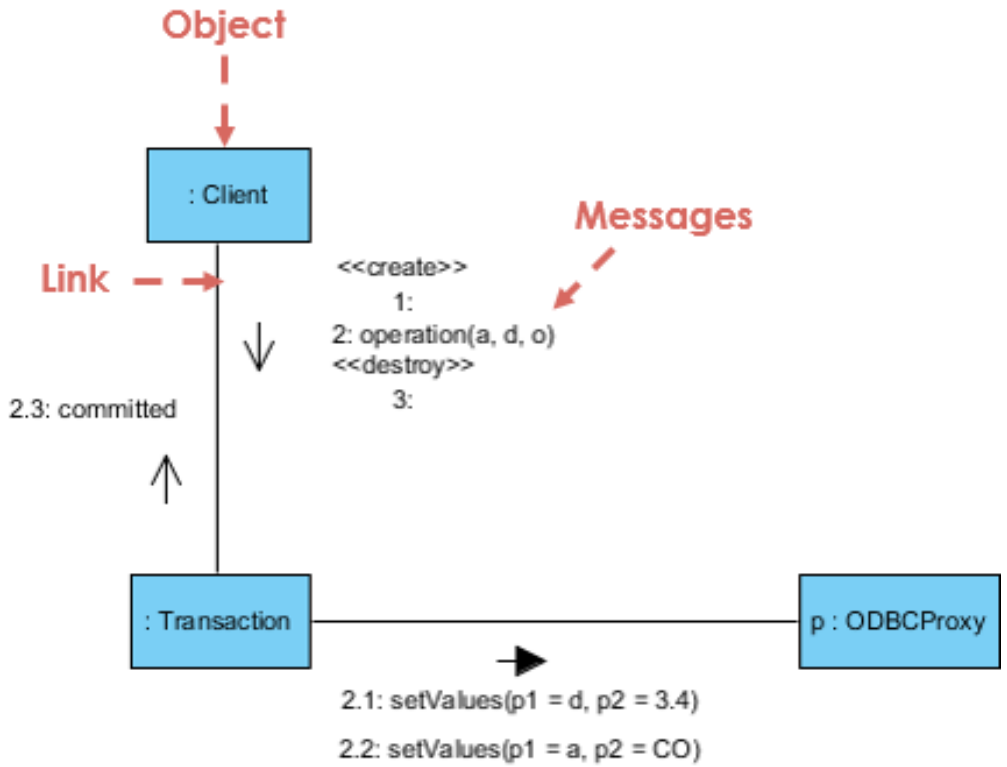
COMMUNICATION DIAGRAM SYMBOLS AND NOTATIONS

- The symbols and notations used in communication diagrams are the same notations for sequence diagrams.

- Rectangles represent objects that make up the application.
- Lines between class instances represent the relationships between different parts of the application.
- Arrows represent the messages that are sent between objects.
- Numbering lets you know in what order the messages are sent and how many messages are required to finish a process.

SEQUENCE DIAGRAM VS COMMUNICATION DIAGRAM





COMMUNICATION DIAGRAM

- *Example: Hotel Reservation*